

DEVALIVE

Engineering Standards & Security Document (ESSD)

Version: 1.0

Document Status: Approved

Purpose:

This document defines the engineering practices, coding standards, security requirements, naming conventions, logging standards, and development rules that must be followed throughout the DevAlive project.

The objective is to maintain:

- Code Quality
- Consistency
- Security
- Scalability
- Maintainability

across the entire application.

1. Engineering Principles

All development must follow the following principles:

Scalability First

Security By Default

Clean Architecture

Separation Of Concerns

Reusable Components

Maintainable Code

2. Code Organization Standards

Backend Architecture

```
Routes
↓
Controllers
↓
Services
↓
Repositories
↓
Database
```

Rules:

- Routes contain route definitions only.
 - Controllers handle requests and responses only.
 - Services contain business logic.
 - Repositories contain database operations.
 - Models contain schemas only.
-

Frontend Architecture

```
Pages
↓
Components
↓
Hooks
↓
Services
↓
API
```

Rules:

- Pages manage screen-level logic.

- Components remain reusable.
 - Services handle API communication.
 - Hooks contain reusable state logic.
-

3. Naming Conventions

Variables

Use:

```
camelCase
```

Example:

```
projectName  
endpointUrl  
responseTime
```

Functions

Use:

```
camelCase
```

Example:

```
createProject()  
sendAlert()  
calculateUptime()
```

React Components

Use:

PascalCase

Example:

```
ProjectCard.jsx  
AnalyticsChart.jsx  
DashboardLayout.jsx
```

Models

Use:

PascalCase

Example:

```
User.js  
Project.js  
MonitoringLog.js
```

Collections

Use:

lowercase plural

Example:

```
users  
projects  
notifications  
monitoringlogs
```

Constants

Use:

```
UPPER_SNAKE_CASE
```

Example:

```
MAX_PROJECTS  
DEFAULT_INTERVAL  
JWT_EXPIRATION
```

4. File Naming Standards

React Components

```
PascalCase
```

Example:

```
ProjectCard.jsx  
Navbar.jsx
```

Utility Files

```
camelCase
```

Example:

```
formatDate.js  
calculateUptime.js
```

Route Files

```
module.routes.js
```

Example:

```
auth.routes.js  
project.routes.js
```

Controller Files

```
module.controller.js
```

Service Files

```
module.service.js
```

Repository Files

```
module.repository.js
```

5. API Development Standards

All APIs must:

- Follow REST principles
- Return JSON responses
- Use proper HTTP status codes
- Follow versioning

Example:

```
/api/v1/projects
```

Response Format

Success

```
{
  "success": true,
  "message": "Project created successfully",
  "data": {}
}
```

Error

```
{
  "success": false,
  "message": "Validation failed",
  "errors": []
}
```

6. Error Handling Standards

Centralized Error Handling Required.

Flow

```
Route
↓
Controller
↓
Service
↓
Error
↓
Global Error Handler
```

Never

Return Raw Errors To Users

Log Errors Internally.

7. Logging Standards

Logging Categories

Application Logs

Purpose:

General System Events

Monitoring Logs

Purpose:

Health Check History

Error Logs

Purpose:

Exception Tracking

Notification Logs

Purpose:

Alert Delivery Tracking

Future Tools

Winston

Morgan

Cloud Logging

8. Git Standards

Branch Naming

Feature

feature/project-management

Bug Fix

fix/monitoring-timeout

Refactor

refactor/analytics-module

Commit Message Format

feat: add project monitoring

fix: resolve login validation

```
refactor: optimize analytics query
```

9. Security Principles

Security Philosophy

```
Assume Every Request Is Malicious
```

Validate everything.

Trust nothing.

10. Authentication Security

Authentication Method

```
JWT
```

Password Storage

```
bcrypt
```

Never Store:

```
Plain Text Passwords
```

JWT Requirements

```
Expiration Required
```

```
Secret Stored In Environment Variables
```

11. Authorization Security

Users Can Access:

Only Their Own Data

Examples:

Projects

Monitoring Logs

Notifications

Settings

Forbidden:

Cross User Access

12. Input Validation Standards

Validate:

Request Body

Request Params

Query Parameters

Required Validations

Email Validation

URL Validation

ObjectId Validation

Password Validation

Technology

express-validator

13. Rate Limiting Strategy

Protect:

Login

Register

Manual Health Check

Future Password Reset

Technology

express-rate-limit

14. Environment Variable Security

Sensitive Data Must Never Be Hardcoded.

Examples:

JWT_SECRET

MONGODB_URI
SMTP_USER
SMTP_PASSWORD

Stored In:

.env

Never Commit:

.env

Required:

.gitignore

15. Database Security

MongoDB Access Rules

Authenticated Access Only

Sensitive Data

Passwords

Stored As:

bcrypt Hash

Queries Must Be Sanitized

Prevent:

NoSQL Injection

16. API Security

Use:

Helmet

Purpose:

Security Headers

Use:

CORS

Purpose:

Origin Protection

Use:

Request Sanitization

Purpose:

Prevent Malicious Payloads

17. Frontend Security

Protected Routes

Require:

Valid JWT

Unauthorized Users

Redirect To:

/login

Sensitive Information

Never Store:

Passwords

Secrets

API Keys

18. Monitoring Security

Manual Monitoring Endpoint

Must Require:

Authentication

Monitoring Requests

Must Include:

Timeout Handling

Prevent:

Infinite Waiting Requests

19. Dependency Management Standards

Only Add Dependencies When:

Necessary

Avoid:

Duplicate Libraries

Abandoned Packages

Unmaintained Packages

Regularly Update:

Dependencies

Security Patches

20. Code Quality Standards

Rules

Single Responsibility Principle

Reusable Components

Reusable Services

Reusable Utilities

Avoid:

Massive Components

Massive Controllers

Repeated Logic

21. Documentation Standards

Every Module Must Have:

Purpose

Responsibilities

Dependencies

Complex Functions Must Include:

Comments

22. Production Readiness Checklist

Before Production Deployment:

Authentication Tested

Authorization Tested

Validation Tested

Monitoring Tested

Notifications Tested

Environment Variables Configured

Error Handling Verified

Rate Limiting Enabled

Security Headers Enabled

23. Engineering Standards Summary

All DevAlive development must prioritize:

- Security
- Scalability
- Maintainability
- Reusability
- Consistency

This document acts as the engineering rulebook for all frontend, backend, testing, deployment, and future contributors to the platform.